绘制关键点

绘制关键点
- 例程简介
  - 什么是关键点?
- API 文档
  - 寻找关键点
  - 绘制关键点
- 示例代码
- 例程运行效果

例程简介

本节我们介绍绘制关键点的draw_keypoints方法

代码位于 [源码汇总 / 03.Drawing / 07.keypoints_drawing.py]

什么是关键点?

这里的关键点（keypoints）是指图像中的特征点，通常是图像中比较显著或者独特的点，具体来说：

特征点的特性：

- 这些点通常是图像中的角点、边缘交叉点等具有显著特征的位置
- 它们在图像的不同视角下都容易被识别
- 这些点的周围像素值变化较大，使其具有独特性

实际应用中的关键点可能是：

- 物体的角点
- 纹理丰富区域的特征点
- 图案中的显著交叉点
- 物体轮廓上的重要点

用途：

- 目标跟踪
- 物体识别
- 图像匹配
- 运动检测

> 由于寻找关键点对性能消耗较大，所以我们减少了识别的区域，只识别屏幕最中心的部分的关键点

API 文档

寻找关键点

```
image.find_keypoints([roi[, threshold=20[, normalized=False[, scale_factor=1.5[,
max_keypoints=100[, corner_detector=image.CORNER_AGAST]]]]]])
```

该函数从指定的 ROI 元组 `(x, y, w, h)` 中提取 ORB 关键点。您可以使用 `image.match_descriptor` 函数比较两组关键点以获取匹配区域。若未发现关键点，则返回 `None`。

- `roi` 是感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，默认 ROI 为整个图像。操作范围仅限于该区域内的像素。
- `threshold` 控制提取关键点的数量（取值范围为 0-255）。对于默认的 AGAST 角点检测器，该值应设为约 20；对于 FAST 角点检测器，该值应设为约 60 至 80。阈值越低，提取的角点越多。
- `normalized` 是布尔值。若为 `True`，则在多分辨率下关闭关键点提取。若您不关心处理扩展问题，且希望算法运行更快，则将其设置为 `True`。
- `scale_factor` 是一个大于 1.0 的浮点数。较高的比例因子运行速度较快，但图像匹配效果相对较差。理想值介于 1.35 和 1.5 之间。
- `max_keypoints` 是关键点对象能够容纳的最大关键点数量。若关键点对象过大导致内存问题，请适当降低该值。
- `corner_detector` 是提取关键点所使用的角点检测器算法。可选值为 `image.CORNER_FAST` 或 `image.CORNER_AGAST`。FAST 角点检测器速度较快，但准确度较低。

注意： 此方法仅支持灰度图像。

绘制关键点

```
image.draw_keypoints(keypoints[, color[, size=10[, thickness=1[, fill=False]]]])
```

在图像上绘制特征点。

- `color`：指定颜色，适用于灰度或 RGB565 图像。默认为白色。对于灰度图像，可以传递灰度值（0-255）；对于 RGB565 图像，可以传递反向字节序的 RGB565 值。
- `size`：控制特征点的大小。
- `thickness`：控制线条的粗细（以像素为单位）。
- `fill`：如果为 True，则填充特征点。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像

示例代码

```
import sys
import uos as os
import time
from media.sensor import *
from media.display import *
from media.media import *

def init_sensor():
    """
    Initialize camera sensor with specified configuration
    使用指定配置初始化摄像头传感器
    """
    # Create sensor instance with resolution 1280x960
    # 创建分辨率为1280x960的传感器实例
    sensor = Sensor(width=1280, height=960)

    # Reset sensor to default state
    # 将传感器重置为默认状态
    sensor.reset()
```

```
    # Configure channel 1 output format to 640x480 RGB565
    # 配置通道1输出格式为640x480 RGB565
    sensor.set_framesize(width=640, height=480, chn=CAM_CHN_ID_1)
    sensor.set_pixformat(Sensor.RGB565, chn=CAM_CHN_ID_1)

    sensor.set_framesize(width=640, height=480, chn=CAM_CHN_ID_0)
    sensor.set_pixformat(Sensor.GRAYSCALE, chn=CAM_CHN_ID_0)

    return sensor

def main():
    """
    Main function to run camera preview
    运行摄像头预览的主函数
    """
    sensor = None
    roi = (220, 140, 200, 200)  # 从(110,70)开始，宽高都是100像素
    try:
        # Initialize camera sensor
        # 初始化摄像头传感器
        sensor = init_sensor()

        # Initialize virtual display with 640x480 resolution
        # 初始化640x480分辨率的虚拟显示
        Display.init(Display.VIRT, width=640, height=480, to_ide=True)

        # Initialize media management
        # 初始化媒体管理
        MediaManager.init()

        # Start sensor operation
        # 启动传感器运行
        sensor.run()

        # Main loop to capture and display frames
        # 捕获和显示帧的主循环
        while True:
            # Capture frame from channel 1
            # 从通道1捕获帧
            img = sensor.snapshot(chn=CAM_CHN_ID_1)

            img_g = sensor.snapshot(chn=CAM_CHN_ID_0)
            img.draw_rectangle(roi, color=(173, 216, 230), fill=False,
thickness=3)
            keypoints = img_g.find_keypoints(
                threshold=30,
                scale_factor=1.2,
                max_keypoints=30,     # 减少特征点数量
                roi=roi               # 指定ROI区域
            )

            # 如果检测到特征点
            if keypoints:
                print(keypoints)
                # 在图像上绘制特征点
                img.draw_keypoints(
                    keypoints,      # 特征点列表
                    color=(255, 0, 0),  # 红色
```

```
                    size=8,              # 特征点大小
                    thickness=4,          # 线条粗细
                    fill=True            # 填充特征点
                )
            # Display captured frame
            # 显示捕获的帧
            Display.show_image(img)

    except KeyboardInterrupt:
        print("User interrupted the program")
        print("用户中断了程序")

    except Exception as e:
        print(f"An error occurred: {str(e)}")
        print(f"发生错误: {str(e)}")

    finally:
        # Cleanup section
        # 清理部分

        # Stop sensor if initialized
        # 如果传感器已初始化则停止
        if isinstance(sensor, Sensor):
            sensor.stop()

        # Deinitialize display
        # 反初始化显示
        Display.deinit()

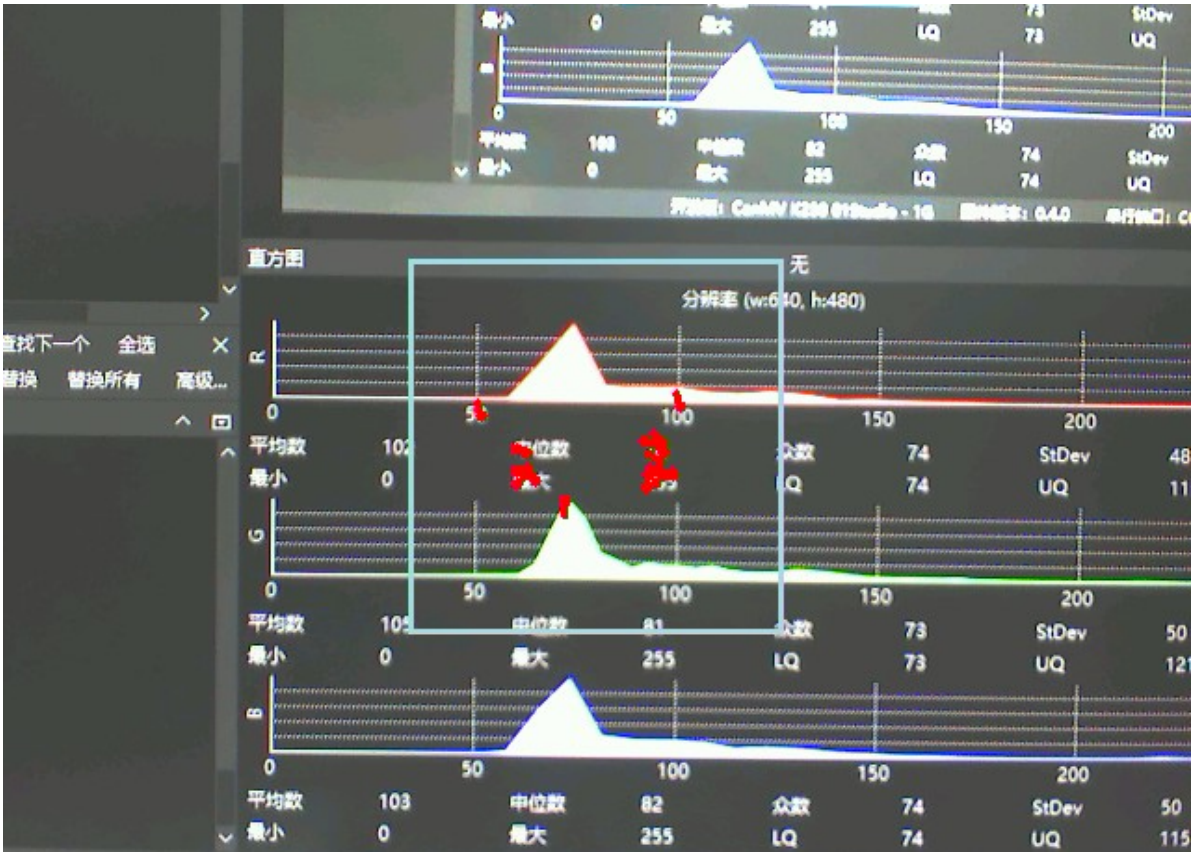
        # Enable sleep mode
        # 启用睡眠模式
        os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
        time.sleep_ms(100)

        # Release media resources
        # 释放媒体资源
        MediaManager.deinit()

if __name__ == "__main__":
    main()
```

例程运行效果

我们用屏幕中间的小框去查看四周环境，会发现画面中的一些关键点被标记出来了


直方图
无
分辨率 (w:640, h:480)
R
G
B
0
50
100
150
200
平均数
中位数
众数
StDev
最小
最大
LQ
UQ
查找下一个
全选
替换
替换所有
高级...